

DocMind — Engineering Build Log

From a Local Prototype to a Cloud-Deployed, Hybrid-Retrieval RAG Architecture

Prepared by Shivam

This log documents the real technical challenges encountered while building and deploying DocMind, a multi-document Retrieval-Augmented Generation (RAG) chatbot, and the engineering decisions made to resolve them. Each entry follows the project's actual chronology — the problem as it first appeared, the root cause once diagnosed, and the solution implemented — with screenshots captured along the way.

Highlights

- **Retrieval failure, wrong first hypothesis:** The document loader was suspected, but inspecting every chunk showed the real cause was a vocabulary mismatch. Fixed with hybrid BM25 + vector retrieval merged by Reciprocal Rank Fusion.
- **A misleading "check your API key" error:** Sources had rendered, so retrieval worked and the failure was in the generation call, most likely a free-tier rate limit or overload. The message was replaced and a log check adopted.
- **Stale vector store after an embedding swap:** Moving from 384 to 768 dimensions failed only at the first insert, because the recovery path wrapped the constructor. The store is now cleared and re-embedded after any model change.
- **Citations taken out of the model's hands:** Citations are built programmatically from the metadata of retrieved chunks and never generated by the model.
- **A zero-inbound deployment on 1 GB of RAM:** A t3.micro with no inbound rules, reached through SSM port forwarding, using hosted embeddings, a 2 GB swap file and no NAT Gateway.

1. Model Pipeline Architecture

Incident: Hugging Face Pipeline Abstraction Failure

- **Challenge:** Initializing the `flan-t5` model via the high-level `pipeline("text2text-generation")` API resulted in a runtime `KeyError`, incorrectly flagging a standard task as "Unknown."
- **Root Cause:** The Hugging Face transformers library failed to map the task string due to a local environment registry initialization failure. The high-level pipeline abstraction relies on dynamic lookups that proved unstable in the active deployment environment.
- **Solution:** Bypassed the unstable `pipeline()` abstraction layer entirely. Refactored the generation module to explicitly instantiate the underlying foundation classes (`AutoTokenizer` and `AutoModelForSeq2SeqLM`), forcing a direct, stable initialization of the local model and resolving the crash.

```
KeyError: "Unknown task text2text-generation, available tasks are ['any-to-any', 'audio-classification', 'automatic-speech-recognition', 'depth-estimation', 'document-question-answering', 'feature-extraction', 'fill-mask', 'image-classification', 'image-feature-extraction', 'image-segmentation', 'image-text-to-text', 'keypoint-matching', 'mask-generation', 'ner', 'object-detection', 'sentiment-analysis', 'table-question-answering', 'text-classification', 'text-generation', 'text-to-audio', 'text-to-speech', 'token-classification', 'video-classification', 'zero-shot-audio-classification', 'zero-shot-classification', 'zero-shot-image-classification', 'zero-shot-object-detection']"
```

Traceback:

```
File "C:\Users\adrak\Desktop\proj5\ragchatbot.py", line 18, in <module>
    gen_tokenizer, gen_model = load_generator()
                                ~~~~~^

File "C:\Users\adrak\Desktop\proj5\ragchatbot.py", line 15, in load_generator
    return pipeline("text2text-generation", model="google/flan-t5-large")
File "C:\Users\adrak\Desktop\proj5\vr\Lib\site-packages\transformers\pipelines
    normalized_task, targeted_task, task_options = check_task(task)
                                ~~~~~^
File "C:\Users\adrak\Desktop\proj5\vr\Lib\site-packages\transformers\pipelines
    return PIPELINE_REGISTRY.check_task(task)
File "C:\Users\adrak\Desktop\proj5\vr\Lib\site-packages\transformers\pipelines
    raise KeyError(f"Unknown task {task}, available tasks are {self.get_support
```

[Copy](#) [Ask Google](#) [Ask ChatGPT](#)

```
@st.cache_resource
def load_generator():
    tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-large")
    model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-large")
    return tokenizer, model
```

2. Provider Volatility

Challenge: Two different Groq model identifiers returned HTTP 404 errors — "model not available" — despite appearing in Groq's own published documentation.

Root Cause: Per-account model availability does not always match published documentation, particularly as providers move models between access tiers.

Solution: Replaced guesswork with a direct diagnostic: queried the account's own model-listing endpoint to see exactly which models were actually accessible, rather than iterating through documented model names by trial and error.

```
1  import os
2  from dotenv import load_dotenv
3  from groq import Groq
4
5  load_dotenv("api.env")
6  client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
7
8  models = client.models.list()
9  for m in models.data:
10     print(m.id)
```

3. Document Ingestion & Metadata Engineering

Challenge: The vector store rejected documents during ingestion with a metadata validation error, caused by a nested dictionary (bounding-box coordinate data) attached by the document parsing library.

Root Cause: The vector database's metadata schema only accepts flat primitive types (string, integer, float, boolean, or null) — nested structures are not supported.

Solution: Applied a metadata-filtering step before every insertion, stripping non-primitive fields while preserving every field actually needed downstream (filename, page number, location label).

Challenge: Supporting multiple file formats (PDF, DOCX, TXT, CSV, XLSX) meant no single, uniform way to label where a given chunk of text came from — PDFs have real page numbers, but plain text and Word documents do not.

Solution: Built a per-file-type loader mapping, with each format routed to the extraction library best suited to it. Added a unified **location** metadata field: real page numbers where the source format provides them, falling back to a chunk-sequence-based section label otherwise. Paired this with deterministic hash-based chunk identifiers, so re-uploading an unchanged file updates existing records instead of creating duplicates.

4. Diagnosing a Retrieval Failure

Challenge: A direct factual query against an uploaded document returned no usable answer, even though the relevant information was clearly present in the source text.

Root Cause: Initial investigation suspected the document loader's content-categorization logic was excluding the relevant text from what got embedded. Direct inspection of every chunk's category and content ruled this out — the text was present and embedded correctly. The actual cause was a vocabulary mismatch: the embedding model couldn't reliably connect a literal query term to a chunk of text that never contained that exact word.

Solution: Rebuilt retrieval as a hybrid system — a keyword-based (BM25) retriever combined with the existing semantic vector retriever, merged via Reciprocal Rank Fusion. This closed the gap between literal keyword queries and semantically-phrased queries, each compensating for the other's weak spot.

This was a deliberate exercise in root-cause discipline: the first hypothesis was plausible but wrong, and only direct inspection of intermediate output — rather than assuming the first likely cause — surfaced the real issue.

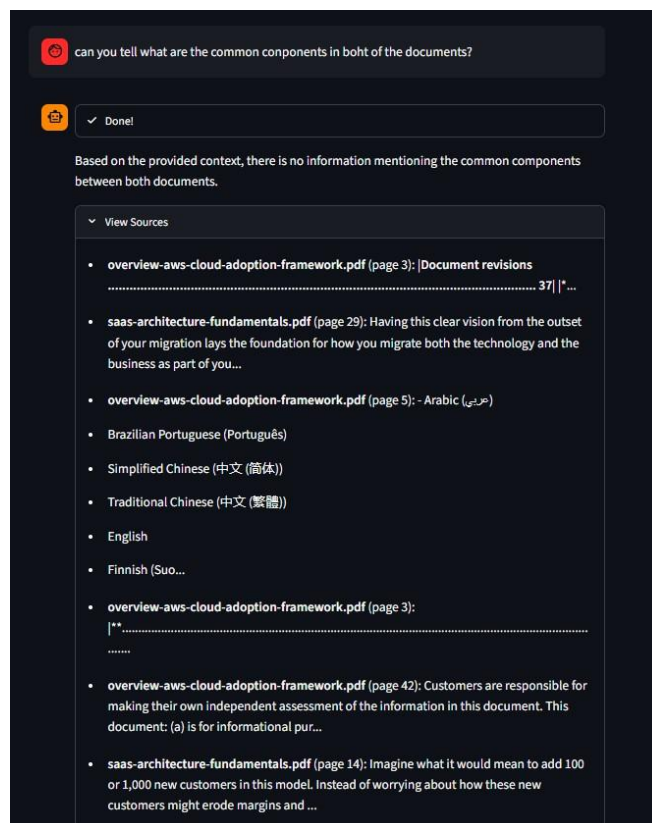
5. Hybrid Retrieval & Search Quality

Challenge: The first custom keyword-retriever implementation returned raw text strings instead of structured document objects, silently stripping away all metadata. Every result sourced from this retriever showed its citation as "unknown."

Solution: Rebuilt the keyword index on integer positions rather than raw text, mapping scored results back to the original document objects by index — preserving full metadata end-to-end through the retrieval pipeline.

Challenge: Combined (ensemble) retrieval was returning several near-duplicate chunks instead of diverse, useful context, and in testing, low-value boilerplate — tables of contents, multi-language index lists — was surfacing as cited evidence for an answer.

Solution: Enabled Maximal Marginal Relevance (MMR) on the vector retriever to actively favor diverse results over redundant ones, added a deduplication pass keyed on source document and chunk content, and added structural-noise filtering to the preprocessing step.

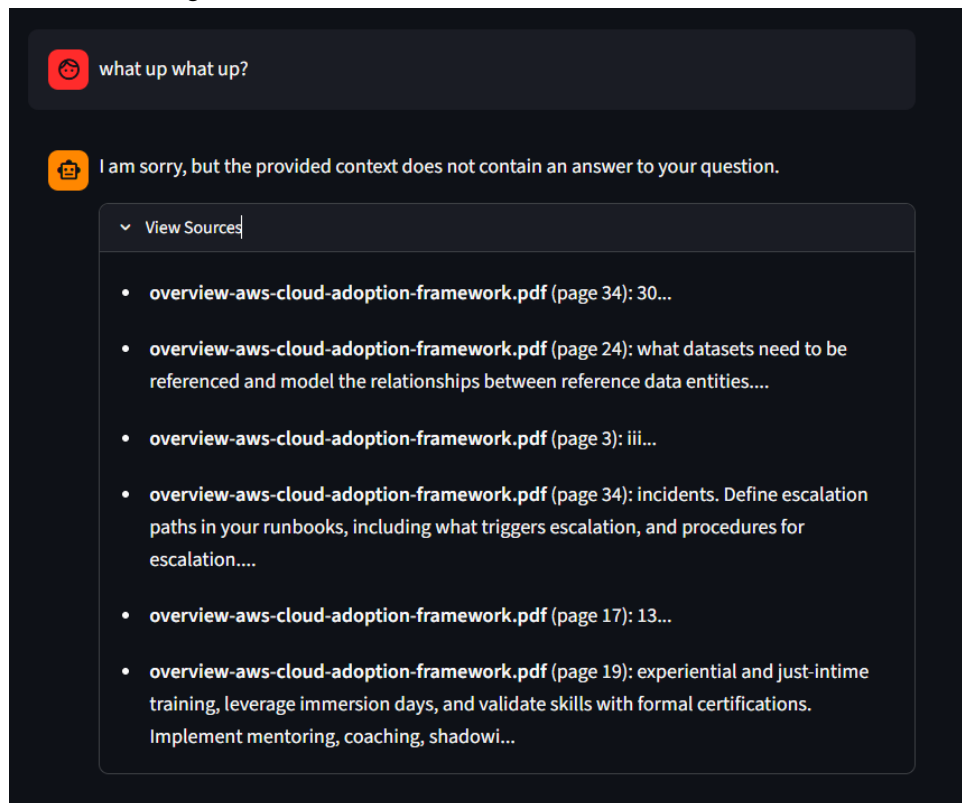


Early testing: sources panel surfacing table-of-contents and language-list boilerplate instead of substantive content.

Challenge: A refusal answer ("the provided context does not contain an answer") was still displaying a full sources list underneath it, misleadingly implying evidence existed.

Solution: Added a check so the sources panel only renders when the answer actually draws on

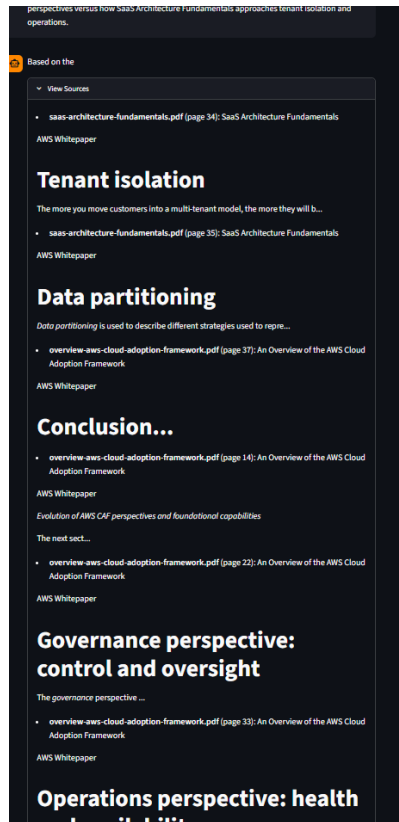
retrieved content, not alongside a refusal.



Bug caught in testing: a refusal answer still rendering a populated sources list beneath it.

Challenge: Deciding when retrieval confidence was too low to trust required a reliability signal, and relying on a single score (from only one retrieval method) proved fragile.

Solution: Built a dual-signal confidence check requiring both the keyword-search score and the vector similarity distance to independently indicate a weak match before triggering a more expensive query-expansion fallback — avoiding both over-triggering and under-triggering from any single weak signal.



Cross-document retrieval working as intended: sources cited from two different uploaded PDFs in a single answer.

6. LLM Output Reliability & Source Attribution

Challenge: Structured output generation intermittently failed with a tool-invocation error, specifically on short refusal-style answers such as "I don't have that information in the documents."

Root Cause: The model occasionally generated a direct text answer instead of invoking the expected function-call wrapper required by strict tool-choice mode. The identical failure mode later recurred independently in the summarization pathway, traced to the same underlying cause.

Solution: Switched structured output generation to JSON mode, paired with an explicit instruction in the prompt itself requesting the exact field name expected — rather than relying on the schema definition alone to carry that constraint.

Challenge: Allowing the model to generate its own source citations alongside its answers introduced a real hallucination risk — a plausible-but-incorrect citation is difficult to catch.

Solution: Decoupled citation generation from the model entirely. Citations are now assembled programmatically, built directly from the metadata of whichever chunks were actually retrieved for that query — never generated by the model itself.



What are the core perspectives and phases outlined in the AWS Cloud Adoption Framework?



[[{"type": "text", "text": "Based on the provided context, the AWS Cloud Adoption Framework (AWS CAF) includes six perspectives, but only one is explicitly named in the text:\n\n* People perspective: culture and change\n\nThe context does not provide the names of the other five perspectives, nor does it contain information about the phases outlined in the framework.", "signature": "'signature':\n\n'Eu4NCusNAWkUfROJsVQ7Jd6eqp9s7bixol/QI7zmlyv30H32s96Ung7q2a44lf8u8l+oK5HgSSkPJv3CJLvQLLrNDLB5DvQNx5CDHK+rVUDfABY4cBW1WWRRPNMDXeVBLE4HkQDhvdh8xoKxtDwlT9cbhbOxxj7HNUEUHSipCyPm2Ck6l9xCeX5WPLCNXBxqu8/LdHFGGgiKE/IQtwLuxRnwPm7lllv4YlIBn+go+WDO8Dww8M+hIYZ5ivkQrjyY9G9pxf9EjF6FqRSmVNHqs1uhmTmD0oODMmSvVoYJ6VoJ9r4L7/F1WX5NlJQ9QB6JxPSTHxbl81BGaePKiKdM5Sfjhut0gyyH5s6D0P1gGmBiZpbTqCs1Sas3Fi+x35Oj8HW+yKWTx30VHKQdfwY4H27KJixKj389YF8QZZvub2F079nzYQkyB9/jhddrl6OCBQcptWGmF1B4o4fW7f/ku/ngnuX8z5rFz6SpZdQRCCOiRoIVFuzVSqJdWEGM2B/dC/1lYtENKGASwOcG6RqdN3uXr+EoRiAELid6dr0aN6thj0Ggs3p4Rx2RHeKxs0M9i8yFkKwpc9a98Or/CQbQnEVj+04fBheX7i7JoiwtDgTg3lyNwZWYOva8v/TW0VIIiNHrd2bEZagnRWJDDbHHcWpliWuyxXpw54CDboDdDeJiUAF/Q2Z1tWnnrrM8oINTF8jelYoNSSUrVYBTLcLxiZcMKV62sQZnfFs4913m5SidsVfGAtWg0Bq0tcYTOo+ynuJduluiKmxteAZXc5M0cSg4ihrsdqoshELjY6tRy7HabBGVynlbO+z1QojSVsjXk4GcS5HpHzGpGYdXN67eldh7ODl3/HA9l6OfiFpAzh3qDJTaff//75tex+7UTd95q5noGRA5pNezhbSoE83U3N/oJgZpGnd/i7gMwJ9GMnsvKkdGoTWSMNAM03Y3bghRT3ialAXkGwBgXlI4IFH45a/vtgVx6CpV17feywqwCrzwgP2R2dvFB7gmC/cyLeu+HPmmelvEN5CLihaFk+yN3lOyHMoM+cz7d63FG02YbeLbVsfE+u9XrjUrEp76Wgp0JYOWbGGlliXeOupX5n71t43qUy82M8amnr5SRnll0IW5xdO2Q+2incQAwX9rVZBHgy/qUBQznqXNAyrGpd9cNsw"}]]

A later recurrence of output-parsing fragility after a provider swap: raw content blocks and an internal signature leaking directly into the chat UI instead of clean text.

This is the same underlying class of problem as the original structured-output failures: whenever the generation provider changes, the assumption that a response is always a plain string has to be re-verified, not re-used.

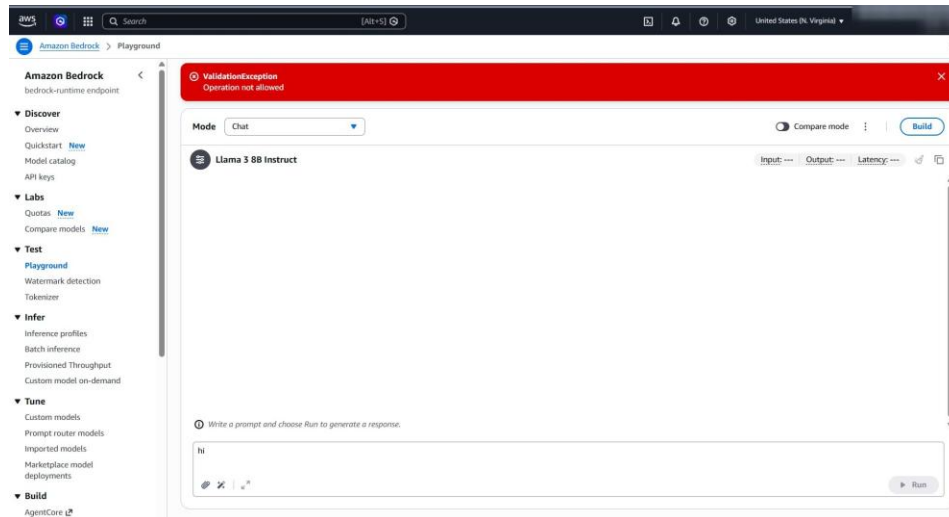
7. A Deliberate Scope Correction

Challenge: The project had organically grown into a general-purpose agent — adding live web search as a fallback, general-knowledge answers outside the uploaded documents, and score-based routing between "answer from documents" and "search the web."

Solution: Recognized this diluted the project's core value. A correctly-grounded, document-only RAG system — one that reliably refuses to answer outside its source material rather than guessing — is a stronger, more defensible engineering demonstration than a loosely-scoped general assistant. The web-search layer was removed entirely, and effort was redirected toward grounding quality and retrieval precision instead.

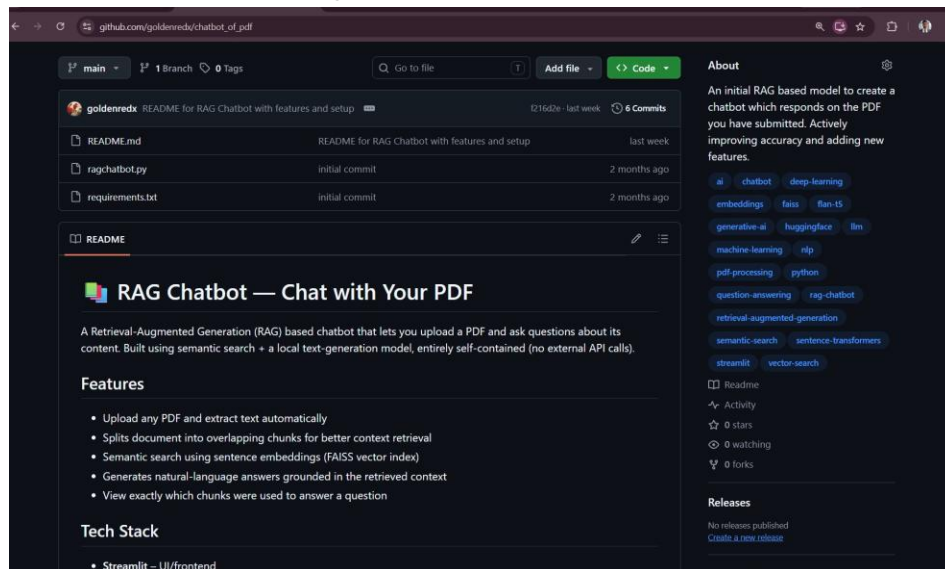
8. Chasing Model Access Across Cloud

Every provider gated access differently: Bedrock rejected the first Llama 3 8B call with a `ValidationException` ("Operation not allowed"), and Google AI Studio put its newest Gemini model behind a plan upgrade. With Bedrock access still blocked, the chat model was swapped to Gemini.

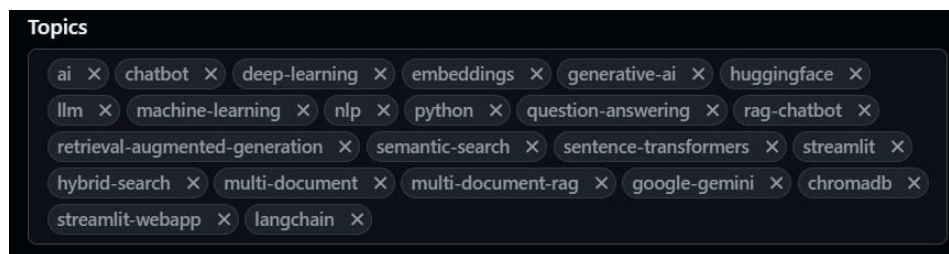


First Bedrock attempt: a `ValidationException` — "Operation not allowed" — calling Llama 3 8B Instruct in the Playground.

The project's own identity shifted alongside these provider changes — visible in how the GitHub repository evolved from its first working version:



The original repository: a single-PDF chatbot built on `flan-t5`, `FAISS`, and `sentence-transformers` — the very first working version of this project.



The same repository's topic tags weeks later — `hybrid-search`, `multi-document-rag`, `chromadb`, `langchain`, `google-gemini` — a visible record of how far the scope had moved.

9. Moving Embeddings to a Hosted API

Challenge: Replacing the locally loaded embedding model with Hugging Face's hosted inference endpoint changed more than one line of configuration. The embedding library's default credential variable (HUGGINGFACEHUB_API_TOKEN) differed from the name already used in the project's environment file (HUGGINGFACEHUB_API_KEY), a mismatch that would only surface as an authentication failure at the first embedding call.

Solution: Passed the token explicitly from the project's own variable and kept the secrets in one place (Parameter Store, described in the deployment section below) instead of scattering them across environment files.

Challenge: Changing the embedding model changes the vector dimension (384 to 768 for the model chosen), but the persistent vector store from earlier runs was still on disk. The store's recovery path wrapped only its constructor, and a dimension mismatch is not raised there; it appears later, at the first insert.

Root Cause: The vector store persists across restarts, whereas the keyword index and the uploaded-file registry are rebuilt for every session. Stale vectors from earlier files could therefore be returned by the semantic retriever without the keyword retriever knowing they exist.

Solution: Treated the on-disk store as disposable: cleared it before each fresh start and re-embedded everything after any model change, instead of relying on exception handling to detect incompatibility.

10. Deploying on AWS Free Tier with Session Manager

The deployment targets a single free-tier-eligible t3.micro instance with no inbound access at all.

Challenge: Enterprise templates default to private subnets and a Managed NAT Gateway, which is unnecessary cost for a proof of concept.

Root Cause: A NAT Gateway bills roughly \$32+/month even when idle, and AWS now charges hourly for every public IPv4 address, so a leftover Elastic IP keeps billing between test sessions.

Solution: Skipped the NAT Gateway and used the instance's auto-assigned public IP for outbound calls to Gemini and Hugging Face. No Elastic IP is attached, so nothing keeps billing while the instance is stopped.

Challenge: Reaching the Streamlit interface without opening any inbound port. The browser-based Session Manager terminal provides a shell, but it cannot forward a port to the local machine.

Solution: Used an SSM port-forwarding session from the local machine (AWS CLI v2, the Session Manager plugin and an authenticated CLI session) to carry localhost:8501 to the instance. Streamlit binds only to 127.0.0.1 and the security group has no inbound rules, so the public IP the instance needs for outbound API calls exposes nothing.

```
C:\Users\adrak>aws ssm start-session --target [REDACTED] --region us-east-1 --document-name AWS-StartPortForwardingSession --parameters "{\"portNumber\":[\"8501\"],\"localPortNumber\":[\"8501\"]}"

Starting session with SessionId:[REDACTED]
Port 8501 opened for sessionId:[REDACTED]
Waiting for connections...
```

SSM port forwarding: local port 8501 is tunnelled to the instance while no inbound rule is open.

Challenge: A 1 GB t3.micro had to install and run a heavy dependency set (document parsers, vector store, LangChain packages), and a locally loaded embedding model on top of that risked exceeding memory under real load, a risk invisible at idle.

Solution: Kept the stronger embedding model but called it through a hosted inference API instead of downsizing it, so no model weights or torch load into memory. Added a 2 GB swap file persisted through /etc/fstab and installed with pip --no-cache-dir.

Challenge: Needed encrypted, access-controlled storage for API keys without recurring cost.

Solution: Chose SSM Parameter Store (SecureString, IAM-gated, no charge) over a managed secrets service for this project's scale.

Parameter Store setup (values redacted) and the least-privilege policy attached to the instance role:

```
# Store each secret as an encrypted SecureString (run once, from a trusted machine)
aws ssm put-parameter --name /docmind/GOOGLE_API_KEY \
  --type SecureString --value "<redacted>" --region us-east-1
aws ssm put-parameter --name /docmind/HUGGINGFACEHUB_API_KEY \
  --type SecureString --value "<redacted>" --region us-east-1

# Read at runtime on the instance (allowed only for /docmind/*)
export GOOGLE_API_KEY=$(aws ssm get-parameter \
  --name /docmind/GOOGLE_API_KEY --with-decryption \
  --region us-east-1 --query Parameter.Value --output text)

# Inline IAM policy on the instance role
{
  "Effect": "Allow",
  "Action": ["ssm:GetParameter", "ssm:GetParameters", "ssm:GetParametersByPath"],
  "Resource": "arn:aws:ssm:us-east-1:<account-id>:parameter/docmind/*"
}
```

Challenge: Processes did not survive the session. Refreshing the browser tab ended the shell, and the new session had no virtual environment activated and no exported API keys.

Solution: Re-ran the pip install in the new session (pip reports "Requirement already satisfied" when the installation is complete) and started the app with setsid nohup, so it is detached from the shell session.

Challenge: A hidden cost default: T3 instances launch with CPU credit specification set to Unlimited, which bills surplus CPU credits outside the free tier.

Solution: Switched it to Standard, which throttles under sustained load instead of billing, and reviewed the remaining advanced options (detailed monitoring left off, IMDSv2 required, no Elastic IP).

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Linux base pricing: 0.0104 USD per Hour

On-Demand Windows base pricing: 0.0196 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour

On-Demand SUSE base pricing: 0.0104 USD per Hour

On-Demand RHEL base pricing: 0.0392 USD per Hour

☐ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

Credit specification [Info](#)

Unlimited ▲

Select

Unlimited ✓

Standard

IAM instance profile [Info](#)

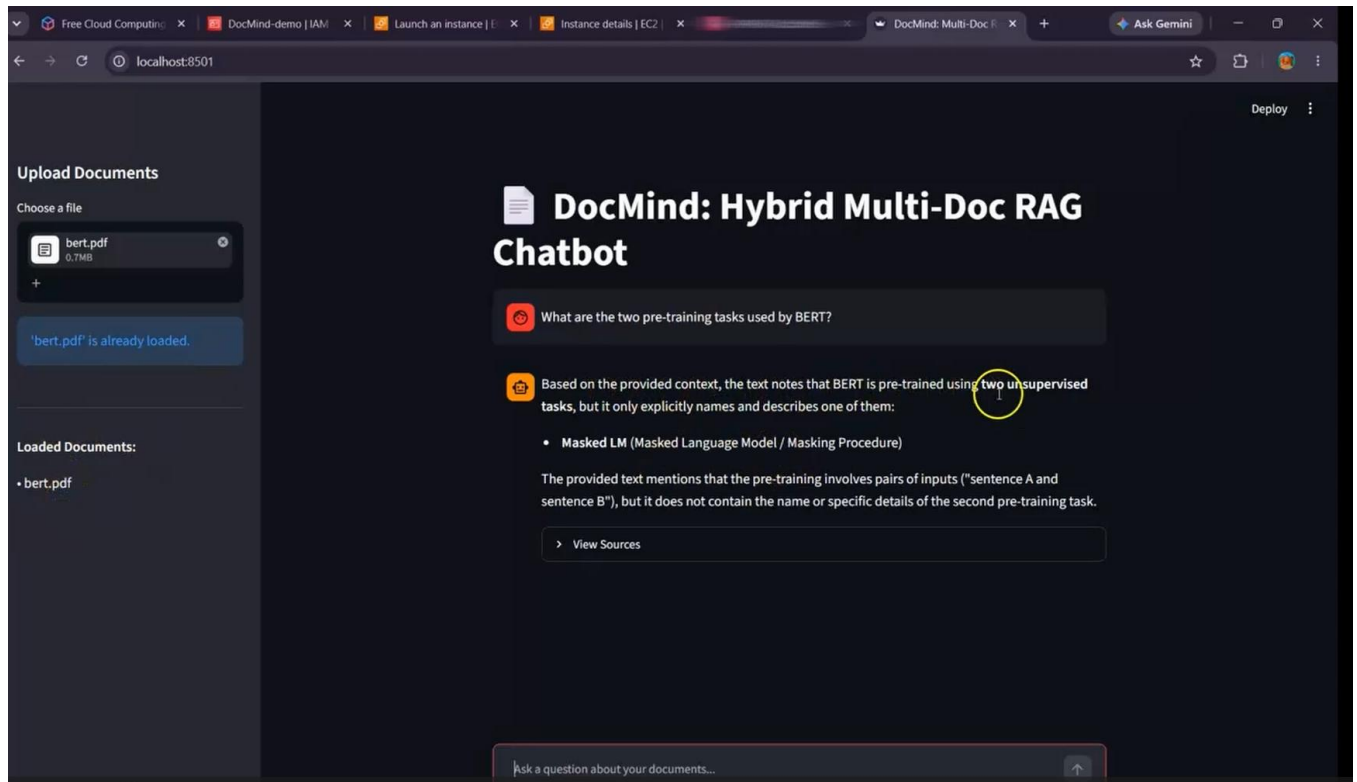
DocMind-demo

arn:aws:iam::[REDACTED]:instance-profile/DocMind-demo ▼

▼ Configure storage [Info](#)

1x GiB Root volume, 3000 IOPS, Not encrypted

Launch wizard: credit specification changed from Unlimited to Standard before launching.



The deployed application, reachable only through the SSM tunnel.

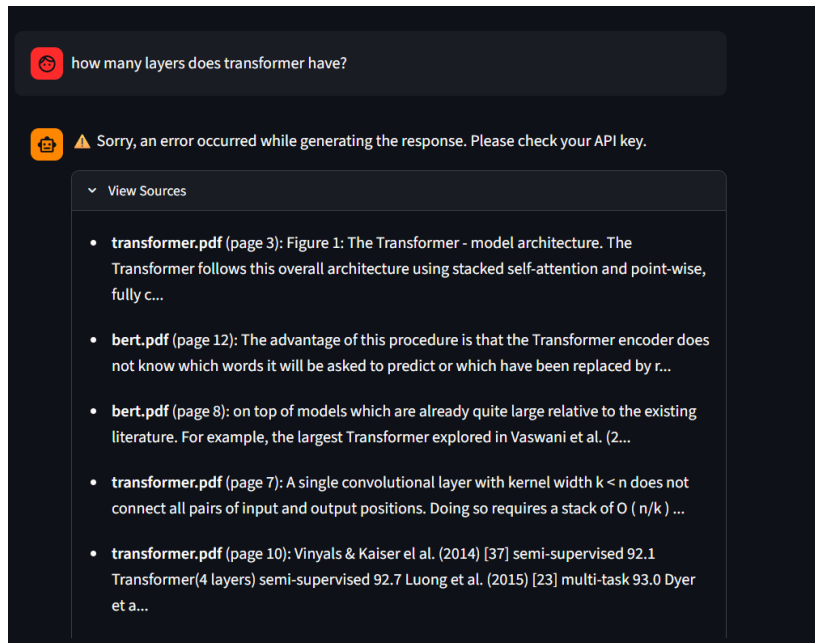
11. What Testing the Deployed App Exposed

Testing on the deployed instance surfaced two issues worth recording.

Challenge: One question returned “Sorry, an error occurred while generating the response. Please check your API key,” while later questions succeeded with the same key.

Root Cause: The sources panel had rendered, so retrieval had worked and the failure sat in the generation call, not the credential. The failing question was a low-confidence one, which triggers the multi-query fallback and an extra LLM request, making a transient rate limit or provider overload on the free tier the most likely cause.

Solution: Replaced the misleading message with a neutral retry message, and adopted a diagnostic step (search the application log for the error class) to separate rate-limit and overload errors from real credential errors.



Retrieval succeeded (sources shown) while the generation call failed.

Challenge: A factual answer that lives in a PDF table (the layer count of the base model in a comparison table) could not be retrieved.

Root Cause: Table cells are flattened into text and then cut apart by the 500-character splitter, separating values from the headers that give them meaning.

Solution: Recorded it as a known limit instead of hiding it; evaluation questions were reworded to target prose statements, and table-aware chunking was added to the roadmap.

12. Cost Control and Teardown

- Stopping keeps the disk (and its charges); terminating deletes it.
- Order: close the tunnel, terminate the instance, confirm no orphaned volume, then delete the Parameter Store entries, IAM role and security group.
- Finish with aws logout; closing the tunnel window does not end the CLI session.

13. Outcome and Next Steps

DocMind now runs as a document-only, hybrid-retrieval RAG system on a single free-tier instance, reached only through an SSM tunnel, with secrets held in Parameter Store and hosted APIs doing the heavy computation.

Final architecture

- **Retrieval:** BM25 and Chroma (MMR) merged by an ensemble retriever, with a confidence-gated multi-query fallback.
- **Models:** BAAI/bge-base-en-v1.5 embeddings through the Hugging Face Inference API; Gemini for answers and summaries.
- **Answers:** grounded only in retrieved context, with citations built from chunk metadata

(filename and page or section) and a bounded three-turn memory.

- **Infrastructure:** t3.micro with zero inbound rules, SSM port forwarding, Parameter Store for secrets and a 2 GB swap file.

Next steps

- Table-aware chunking so numeric table cells keep their column headers.
- LLM-based query rewriting for follow-ups that share no keywords with the previous question.
- A small evaluation set (15 to 20 questions with known answers) to measure retrieval accuracy as settings change.
- A reranking step after retrieval to improve questions that span several documents.

Executive Summary: Architecture & Deployment

The Objective: DocMind is a multi-document Retrieval-Augmented Generation (RAG) chatbot that transitioned from a local prototype into a cloud-deployed, cost-optimized system. The application is strictly scoped to ground its answers exclusively in uploaded context, decoupling citations from the generation model to prevent hallucinations by programmatically assigning source metadata (filename and page/section).

Infrastructure & Security: Deploying on an AWS Free Tier t3.micro instance introduced a strict 1GB RAM hardware constraint. To achieve a near-zero recurring cost, the architecture avoids expensive Managed NAT Gateways. The instance operates in a public subnet but maintains a strict zero-inbound security posture with no open ports; all access is routed securely through an AWS SSM port-forwarding tunnel. API keys are secured via AWS Systems Manager Parameter Store with least-privilege IAM policies. To survive the heavy dependency installation without maxing out storage or memory—because standard pip caches downloads and effectively stores the massive packages twice—the system relies on a 2GB swap file and strict --no-cache-dir installations.

Hybrid Retrieval Pipeline: To resolve vocabulary mismatch failures where literal query terms did not align with semantic embeddings, the retrieval mechanism was rebuilt as a hybrid system. It merges a BM25 keyword index with a Chroma vector store utilizing Maximal Marginal Relevance (MMR) for deduplication and diversity. Retrieval reliability is governed by a dual-signal confidence check, which requires both keyword and vector scores to independently indicate a weak match before triggering a more expensive multi-query expansion fallback.

Model Stack: To preserve high retrieval quality without exceeding the 1GB memory ceiling, local embedding computations were offloaded. The system utilizes BAAI/bge-base-en-v1.5 embeddings through the Hugging Face Inference API, while Google Gemini handles answer generation and summarization.